

Lagersystem

Von der handschriftlichen Inventur zur privaten Cloud-Anwendung

Projektbericht

Migration eines Excel/VBA-Lagerverwaltungssystems auf eine moderne Web-Anwendung

Stand: 16. August 2026

Inhaltsuebersicht

1. Ausgangslage und Zielsetzung
2. Architekturentscheidung: Weg von Excel
3. Aufbau des Grundsystems
4. Digitalisierung der Bestandsdaten
5. Iterative Weiterentwicklung der Oberflaeche
6. Datensicherheit: Papierkorb und Loesch-Bestaetigung
7. Produktionsreife auf dem lokalen Rechner
8. Mehrgeraete-Zugriff und Windows-Vorbereitung
9. Die Entscheidung fuer die Cloud
10. Technische Umsetzung der Cloud-Migration
11. Go-Live: GitHub, Neon und Streamlit Community Cloud
12. Fehlerbehebung nach dem Go-Live
13. Aktueller Stand und offene Punkte

1. Ausgangslage und Zielsetzung

Ausgangspunkt des Projekts war ein bestehendes, handgepflegtes Lagerverwaltungssystem fuer Metall-Lagerbestand (Flacheisen, Rundmaterial, Stahlsorten wie ST37K), das als Excel-Arbeitsmappe mit VBA-Makros („Lager System V3.3.xlsm“) betrieben wurde. Das System bildete Artikelgruppen, Artikel-Stuecke (volle Stangen und Reststuecke), Materialentnahmen sowie ein Verlaufsprotokoll ab und wurde ueber mehrere VBA-Module (u. a. Artikelgruppe, Artikel-Stueck, Export, E-Mail-Versand, Einstellungen) realisiert.

Der erste Arbeitsschritt bestand darin, dieses bestehende System vollstaendig zu verstehen: alle Excel-Tabellen (ListObjects), UserForms und VBA-Module wurden im Detail durchgearbeitet, um die fachliche Logik (insbesondere die Barcode-Vergabe und die Schrott-/Rest-Logik bei Materialentnahmen) korrekt in eine neue Anwendung uebertragen zu koennen.

Anschliessend wurde ueber Alternativen zur Weiternutzung von Excel diskutiert. Nach Abwaegung verschiedener Optionen fiel die bewusste Entscheidung, **nicht** bei Excel zu bleiben, sondern das System als eigenstaendige Web-Anwendung mit einer robusteren Datenbank im Hintergrund neu aufzubauen – unter der ausdruecklichen Vorgabe, die Loesung **so einfach wie moeglich** zu halten und das Projekt **Schritt fuer Schritt** gemeinsam zu entwickeln.

2. Architekturentscheidung: Weg von Excel

Als Zielarchitektur wurde eine Kombination aus **Python**, dem Web-App-Framework **Streamlit** und einer **SQLite**-Datenbank gewaehlt. Diese Kombination erfuellte die Anforderung nach Einfachheit besonders gut:

- Streamlit erzeugt aus reinem Python-Code eine vollstaendige Web-Oberflaeche – kein separates Frontend-Framework, kein manuelles HTML/CSS/JavaScript noetig.
- SQLite speichert die gesamte Datenbank in einer einzigen Datei, ganz ohne separaten Datenbankserver – ideal fuer ein kleines, aber wachsendes Team.
- Die App laeuft als lokaler Webserver und ist von jedem Geraet im selben Netzwerk per Browser erreichbar, ohne dass irgendetwas installiert werden muss.

Von Beginn an wurde die Anwendung in drei klar getrennte Schichten aufgeteilt, die sich durch das gesamte Projekt hindurch bewahrt haben:

app.py	Streamlit-Oberflaeche: alle Seiten, Formulare, Dialoge und die Navigation.		
logic.py	Fachliche	Geschaeftslogik:	Barcode-Vergabe, Materialentnahme-Logik, Suche, Export.
db.py	Datenzugriffsschicht:	kapselt saemtliche	SQL-Zugriffe hinter generischen Funktionen.
schema.sql	Das Datenbankschema, 1:1 an die urspruenglichen Excel-Tabellen angelehnt.		

3. Aufbau des Grundsystems

3.1 Datenmodell

Das Datenbankschema wurde bewusst nah an der Struktur der urspruenglichen Excel-Tabellen gehalten, um die fachliche Logik moeglichst unveraendert uebernehmen zu koennen: **Lager** (Lagerorte), **Typ**, **Einheit**, **Profil_Abk** und **Material_Gruppe** als Stammdaten, **Artikel_Gruppe** als Artikel-Stammsatz (erzeugt den Haupt-Barcode), sowie die drei identisch aufgebauten Tabellen **Artikel_Liste** (volle Stuecke), **Reste** (kurze Stuecke) und **Verlauf** (unveraenderliches Buchungsprotokoll).

Das Barcode-Schema unterscheidet zwischen dem **Haupt-Barcode** einer Artikelgruppe (z. B. „FLS370001“, zusammengesetzt aus Profil- und Material-Kuerzel plus laufender Nummer) und dem **Stueck-Barcode** eines einzelnen physischen Stuecks (z. B. „FLS370001-2“). Damit laesst sich sowohl ein gescannter Gruppen-Code als auch ein gescannter Stueck-Code eindeutig zuordnen.

3.2 Materialentnahme-Logik

Kernstueck der fachlichen Logik ist die Materialentnahme: Wird von einem Stueck eine bestimmte Laenge abgeschnitten, berechnet das System automatisch die Restlaenge, bucht die Entnahme mit negativer Laenge ins Verlaufsprotokoll und legt das Reststueck automatisch in der passenden Tabelle an – liegt die Restlaenge unterhalb einer konfigurierbaren Schwelle (spaeter einstellbar, Startwert 1000 mm), wandert es in **Reste**, sonst bleibt es ein voller Artikel in **Artikel_Liste**. Existiert bereits ein identisches Reststueck (gleicher Haupt-Barcode und gleiche Laenge), wird stattdessen dessen Menge erhoehrt, statt einen weiteren Datensatz anzulegen.

3.3 Mehrbenutzerbetrieb

Da mehrere Personen im selben Netzwerk gleichzeitig auf die Anwendung zugreifen sollten, wurde die SQLite-Datenbank im **WAL-Modus** (Write-Ahead-Logging) betrieben, wodurch sich lesende und schreibende Zugriffe nicht gegenseitig blockieren. Die Vergabe neuer Barcodes wurde zusaetzlich durch einen anwendungsinternen Lock abgesichert, damit zwei gleichzeitige Nutzer nicht denselben Barcode erzeugen koennen. Die Robustheit wurde mit einem Belastungstest von 100 gleichzeitigen Schreibzugriffen (5 parallele simulierte Nutzer) verifiziert – ohne Konflikte oder doppelte Barcodes.

3.4 Export und Kommunikation

Analog zu den urspruenglichen Excel-Funktionen wurden ein Excel-Export, ein PDF-Export (fpdf2) sowie ein E-Mail-Versand per **mailto**-Link (ohne Speicherung von SMTP-Zugangsdaten – der Nutzer haengt die zuvor heruntergeladene Datei manuell an) umgesetzt.

4. Digitalisierung der Bestandsdaten

Die realen Lagerbestände lagen zunächst nur als handschriftliche Notizen vor (vier Fotos handbeschriebener Listen). Um Fehler bei der Übertragung zu vermeiden, wurde zunächst **keine** direkte Übernahme in die Produktivdatenbank vorgenommen. Stattdessen wurde aus den Fotos eine Kontroll-Tabelle („Bestandsaufnahme_Kontrolle.xlsx“) erstellt, die alle erkannten Werte samt unsicherer/unleserlicher Stellen auflistete.

Diese Kontroll-Tabelle wurde vom Auftraggeber vollständig durchgesehen und korrigiert (inklusive expliziter Kennzeichnung unbekannter Materialangaben mit „---“). Erst auf Basis der final bestätigten Tabelle wurden die Daten per Importskript („import_bestand.py“) in die echte Datenbank übernommen: **86 Artikel-Datensätze** über 50 Artikelgruppen hinweg. Diese 86 Datensätze bildeten von diesem Zeitpunkt an durchgehend die reale Produktivgrundlage, gegen die jede weitere Änderung getestet wurde.

5. Iterative Weiterentwicklung der Oberfläche

Im Anschluss an das Grundsystem folgte eine längere Phase kleinteiliger, nutzerzentrierter Verbesserungen. Jede Änderung wurde einzeln umgesetzt, im Browser getestet und erst danach die nächste angegangen. Die wichtigsten Etappen im Überblick:

5.1 Export- und Downloadfunktionen

Zu jeder Tabellenansicht wurde neben dem eingebauten CSV-Download ein zusätzlicher **PDF-Download-Button** ergänzt, direkt neben dem nativen Symbol-Menü platziert. Später wurde diese Position bei gefilterten/gesuchten Tabellen vereinheitlicht: der PDF-Button steht seither immer oben neben der Ergebnis-Überschrift, nicht darunter.

5.2 Modale Dialoge statt Seitenwechsel

Die häufigsten Aktionen (Hinzufügen, Material entnehmen, Bearbeiten/Löschen, Artikelgruppe anlegen, Materialgruppe anlegen) wurden von eigenen Unterseiten auf modale Popup-Fenster (**st.dialog**) umgestellt, die sich per Knopfdruck öffnen, ohne die Seite zu wechseln. Die Fenster wurden vertikal mittig zentriert, alle Eingabefelder untereinander statt nebeneinander angeordnet, und Bestätigungs-Buttons konsequent rechts (Löschen links, Speichern/Hinzufügen rechts) platziert.

Die sechs ursprünglich getrennten Buttons für „Artikel“ und „Reste“ wurden zu drei gemeinsamen Buttons zusammengeführt: Beim Hinzufügen entscheidet seither die eingegebene Länge automatisch, ob ein Stück als voller Artikel oder als Rest gespeichert wird (mit Live-Hinweistext und manueller Überschriftungs-Möglichkeit).

5.3 Layout- und Bedienbarkeits-Feinschliff

- Tabellen zeigen seither ihre volle Höhe ohne inneren Scrollbalken – die Seite selbst scrollt.
- Die einfache Textsuche wurde zu einer tipp- und scanfähigen Dropdown-Kombobox ausgebaut, ergänzt um einen eigenen Abschnitt „Erweiterte Filter“ mit Mehrfachkriterien (Profil, Barcode, Bezeichnung, Lagerort, Material, Mass, Länge von/bis).

- Die Schrott-/Rest-Laengengrenze (urspruenglich fest im Code verankert) wurde ueber eine neue Konfigurationstabelle einstellbar gemacht und beide vormals getrennten Schwellenwerte (Schrott-Erkennung bei Entnahme, automatische Rest-Zuordnung beim Hinzufuegen) auf eine einzige, gemeinsame Einstellung vereinheitlicht.
- Schnellzugriff-Buttons wurden zunaechst auf allen Einzelseiten dupliziert, spaeter zentral in die Seitenleiste verschoben, damit sie von jeder Seite aus verfuegbar sind und nicht mehr redundant gepflegt werden muessen.
- Zahlreiche kleinere Feinjustierungen: Ueberschriftengroessen, Spaltenanzahl in Formularen, einheitliche Button-Breiten sowie eine eigene Filterleiste direkt auf den Seiten „Artikel Liste“ und „Reste“.

6. Datensicherheit: Papierkorb und Loesch-Bestaetigung

Im Zuge der Vorbereitung auf den produktiven Einsatz wurde ein zentrales Risiko identifiziert: Der „Loeschen“-Knopf im Bearbeiten-Dialog entfernte einen Artikel bislang sofort und endgueltig – bei einer Verwechslung beim Scannen waere das ein potenzieller Datenverlust gewesen.

Zur Absicherung wurden zwei Massnahmen umgesetzt: Erstens erscheint vor jedem Loeschvorgang eine explizite Sicherheitsabfrage („Ja, loeschen“ / „Abbrechen“). Zweitens landet ein geloeschtes Stueck nicht mehr sofort im Nichts, sondern in einer neuen **Papierkorb**-Tabelle samt Loeschzeitpunkt. Von dort kann es entweder mit einem Klick **wiederhergestellt** werden, oder – nach einer zweiten, eigenen Sicherheitsabfrage – **endgueltig** geloescht werden.

7. Produktionsreife auf dem lokalen Rechner

Vor der Uebergabe an den Vorgesetzten wurde die Betriebssicherheit der Anwendung systematisch verbessert. Auf die Frage „was sollte vor der offiziellen Nutzung noch passieren?“ wurden vier Punkte identifiziert und umgesetzt:

7.1 Automatisches Backup

Ein Backup-Skript sichert die Datenbank ueber SQLites **VACUUM INTO**-Befehl in eine eigenstaendige, in sich konsistente Kopie. Zunaechst lief dieses Backup nur beim Start der Anwendung – bei einem durchgehend laufenden Hintergrunddienst waere das Backup dadurch aber nie aktualisiert worden. Diese Luecke wurde spaeter durch einen zusaetzlichen, taeglich zu fester Uhrzeit ausgelosten Backup-Zeitplan geschlossen, unabhaengig von Neustarts. Es werden automatisch die letzten 30 Tage aufbewahrt.

7.2 Automatischer Hintergrunddienst

Um die Abhaengigkeit von einem staendig geoeffneten Terminal-Fenster zu beseitigen, wurde ein macOS-**LaunchAgent** eingerichtet: Die Anwendung startet seither automatisch beim Anmelden am Rechner und startet sich bei einem Absturz selbststaendig neu (verifiziert durch gezieltes Beenden des Prozesses im laufenden Betrieb).

7.3 Umzug aus iCloud Drive

Dabei zeigte sich ein grundlegendes Problem: Der Projektordner lag urspruenglich in iCloud Drive, und macOS verweigert Hintergrunddiensten dort systembedingt den Dateizugriff. Nach Ruecksprache wurde das gesamte Projekt in einen lokalen, nicht synchronisierten Ordner verschoben (die urspruengliche Kopie blieb dabei – nur umbenannt – als Sicherheitsnetz erhalten), die virtuelle Python-Umgebung an der neuen Stelle neu aufgebaut und der Hintergrunddienst entsprechend umkonfiguriert.

7.4 Kurzanleitung fuer den Vorgesetzten

Ergaenzend zur technischen README wurde eine kurze, nicht-technische Anleitung fuer die taegliche Nutzung erstellt: Oeffnen der App, Uebersicht der Seiten, die wichtigsten Aktionen im Schnellzugriff sowie das Vorgehen bei versehentlichem Loeschen.

8. Mehrgeraete-Zugriff und Windows-Vorbereitung

Fuer den Zugriff weiterer Mitarbeitender wurden fertige Verknuepfungen erstellt: eine **.webloc**-Datei fuer macOS sowie eine **.url**-Datei fuer Windows, beide direkt auf die Netzwerkadresse der laufenden Anwendung zeigend – kein Installationsschritt noetig. Fuer iOS/Android wurde die Vorgehensweise „Zum Home-Bildschirm hinzufuegen“ dokumentiert.

Da die urspruengliche Loesung an den privaten Rechner des Praktikanten gebunden war, wurde in einem weiteren Schritt ein vollstaendiges **Windows-Setup-Paket** vorbereitet: ein plattformunabhaengiges Backup-Skript (in Python statt Bash), ein Start-Skript fuer den Hintergrundbetrieb (per Windows-Aufgabenplanung, unsichtbar ueber ein VBS-Wrapper-Skript gestartet), ein manueller Doppelklick-Fallback sowie eine ausfuehrliche Schritt-fuer-Schritt-Anleitung zur Einrichtung auf einem Firmenrechner.

9. Die Entscheidung fuer die Cloud

Bei der Planung der Uebergabe wurde ein entscheidender struktureller Schwachpunkt erkannt: Der Praktikant, auf dessen Rechner die Anwendung lief, wuerde das Unternehmen irgendwann wieder verlassen – ein System, das an eine einzelne Person gebunden ist, waere dann nicht mehr betriebsfaehig.

Zur Loesung wurden zwei Wege gegenueberstellt: Erstens die Verlagerung des Hintergrunddienstes auf einen Firmenrechner (z. B. den PC des Vorgesetzten) – kostenlos, ohne zusaetzliches Sicherheitsrisiko, aber weiterhin auf das Buero-Netzwerk beschraenkt. Zweitens echtes Cloud-Hosting – von ueberall erreichbar, aber mit echten Kosten, notwendigem Zugriffsschutz (die Anwendung hatte bis dahin keinerlei Login) und einem groesseren technischen Umbau, da eine einzelne SQLite-Datei nicht zu den meisten Cloud-Plattformen passt.

Nach Abwaegung beider Optionen fiel die Entscheidung zugunsten des Cloud-Wegs: **GitHub** fuer die Quellcodeverwaltung, **Neon** als kostenlose, gehostete Postgres-Datenbank und **Streamlit Community Cloud** als Ausfuehrungsumgebung mit privatem, einladungsbasiertem Zugriff – alle drei Dienste in ihrer kostenlosen Stufe fuer diesen Anwendungsfall ausreichend.

10. Technische Umsetzung der Cloud-Migration

10.1 Zwei-Wege-Datenzugriffsschicht

Anstatt die lokale SQLite-Loesung zu ersetzen, wurde **db.py** so erweitert, dass sie beide Betriebsarten beherrscht: Ist eine **DATABASE_URL** konfiguriert (lokal ueber eine Umgebungsvariable, in der Cloud ueber die Streamlit-Secrets-Verwaltung), verbindet sich die Anwendung mit Postgres; andernfalls bleibt sie unveraendert beim lokalen SQLite-Betrieb. Damit laeuft exakt derselbe Anwendungscode wahlweise lokal oder in der Cloud, ohne Verzweigungen im restlichen Programm.

Fuer Postgres wurde ein eigenes, syntaktisch angepasstes Schema (**schema_postgres.sql**) erstellt sowie ein Migrationsskript (**migrate_sqlite_to_postgres.py**), das die bestehenden Daten aus der lokalen Datenbank vollstaendig und wiederholbar in die Cloud-Datenbank uebertraegt.

10.2 Herausforderungen bei der Umstellung

Die Migration auf eine andere Datenbank-Engine brachte mehrere subtile, erst durch systematisches Testen gegen eine echte lokale Postgres-Testinstanz aufgedeckte Fallstricke mit sich:

- **Gross-/Kleinschreibung von Bezeichnern:** Anders als SQLite faltet Postgres unquotierte Tabellen- und Spaltennamen automatisch auf Kleinschreibung. Ein eigens gebautes Zeilen-Objekt sorgt seither dafuer, dass Ergebniszeilen – unabhaengig vom Datenbanktyp – exakt die im Schema definierte Schreibweise zeigen, sowohl bei direktem Feldzugriff als auch nach Umwandlung in ein einfaches Python-Dictionary.
- **Cursor-Typ-Konflikt mit pandas:** Ein fuer den eigenen dict-artigen Zeilenzugriff gesetzter Cursor-Typ kollidierte mit der Art, wie die Bibliothek pandas Tabellen aus einer Datenbankverbindung einliest – behoben durch getrennte Cursor-Konfigurationen fuer beide Anwendungsfaelle.
- **Alias-Kollision:** Ein in einer Abfrage verwendeter Kurzname geriet in Konflikt mit einer echten Spalte gleichen Namens in einer anderen Tabelle und lieferte dadurch falsche Werte zurueck.
- **Sequenz-Fortsetzung:** Nach dem Uebertragen bestehender Datensaeetze mit bereits vergebenen IDs musste der interne Postgres-Zaehler fuer automatisch vergebene IDs manuell auf den hoechsten uebertragenen Wert gesetzt werden, um spaetere ID-Kollisionen zu vermeiden.

Jede dieser Korrekturen wurde vor der Veroeffentlichung lokal gegen eine eigens installierte Postgres-Testinstanz mit einer Kopie der echten 86 Datensaeetze nachgestellt und durch einen vollstaendigen Klick-Test aller Seiten und Dialoge verifiziert, bevor sie live geschaltet wurde.

11. Go-Live: GitHub, Neon und Streamlit Community Cloud

Fuer die eigentliche Veroeffentlichung wurden folgende Schritte durchgefuehrt:

- Anlegen eines **privaten** GitHub-Repositorys, Hochladen des Quellcodes (die lokale Datenbankdatei, Backups sowie Zugangsdaten wurden ueber eine .gitignore-Konfiguration bewusst vom Repository ausgeschlossen).

- Einrichten einer kostenlosen **Neon**-Postgres-Datenbank und Uebertragung der 86 echten Artikel-Datensaetze samt aller Stammdaten mittels des vorbereiteten Migrationsskripts.
- Deployment der Anwendung auf **Streamlit Community Cloud**, verbunden mit dem GitHub-Repository, die Datenbankverbindung als geschuetztes Secret hinterlegt.
- Einschraenkung der Sichtbarkeit auf ausgewaehlte, eingeladene Personen („Only specific people can view this app“), sodass der Link nicht oeffentlich zugaenglich ist.

Nach dem Deployment wurde die Live-Anwendung direkt im Browser gegen die echten Daten getestet: Anzeige der 86 Artikel und 50 Artikelgruppen, Anlegen, Bearbeiten, Entnehmen und Loeschen eines Artikels sowie die Papierkorb-Funktion – bei laufender Verifikation, dass der weiterhin parallel aktive lokale Mac-Betrieb davon unberuehrt blieb.

12. Fehlerbehebung nach dem Go-Live

Kurz nach der ersten Veroeffentlichung meldete der Auftraggeber Fehlermeldungen beim Aufrufen der Seiten „Artikel Liste“ und „Reste“ sowie beim Anlegen eines neuen Artikels ueber den „Hinzufuegen“-Dialog. Beide Fehler liessen sich auf dieselbe Grundursache zurueckfuehren – Postgres-spezifische Eigenheiten bei der Gross-/Kleinschreibung – traten jedoch an Stellen auf, die von den urspruenglichen Tests nicht abgedeckt waren (ein positionaler Listenzugriff sowie eine zufaellige Namensueberschneidung eines Abfrage-Alias mit einer echten Spalte).

Beide Fehler wurden durch eine grundlegend robustere Loesung behoben, statt einzelne Symptome zu flicken: Das interne Zeilen-Objekt benennt Spalten nun bereits beim Erzeugen der Zeile korrekt um, sodass sowohl der direkte Feldzugriff als auch eine Umwandlung in ein gewoehnliches Python-Dictionary zuverlaessig die richtige Schreibweise liefert. Die Korrektur wurde erneut vollstaendig lokal nachgestellt (inklusive Nachstellen des exakt gemeldeten Ablaufs), vor der Veroeffentlichung durch einen kompletten Klick-Test aller Seiten und Dialoge bestaetigt und anschliessend live verifiziert, bevor der Vorgang als abgeschlossen gemeldet wurde.

13. Aktueller Stand und offene Punkte

13.1 Was heute funktioniert

- Vollstaendige Abbildung aller urspruenglichen Excel-Funktionen: Artikelgruppen, Artikel-Stuecke, automatische Schrott-/Rest-Logik bei Entnahmen, Verlaufsprotokoll, Suche, Excel-/PDF-Export, E-Mail-Entwurf.
- Papierkorb-Funktion und Loesch-Sicherheitsabfrage als Schutz vor Bedienfehlern.
- Lokaler Betrieb auf dem Mac laeuft weiterhin unveraendert und unabhaengig als Hintergrunddienst mit automatischem taeglichem Backup.
- Cloud-Version unter einer privaten, einladungsbasierten Adresse erreichbar – unabhaengig von jedem einzelnen Geraet oder jeder einzelnen Person, mit denselben 86 realen Artikel-Datensaetzen.
- Fertig vorbereitetes, aber noch nicht zwingend benoetigtes Windows-Setup-Paket als Alternative bzw. Rueckfalloption.

13.2 Bekannte Einschraenkungen

- Die kostenlose Neon-Datenbank pausiert nach ca. 5 Minuten Inaktivitaet und braucht beim naechsten Zugriff einen kurzen Moment zum Aufwachen – im normalen Betrieb kaum spuerbar.
- Automatische Datensicherung erfolgt in der Cloud-Variante durch Neon selbst (Point-in-Time-Recovery, im kostenlosen Plan fuer die letzten 6 Stunden) und ersetzt dort das lokale Backup-Skript.
- Die Netzwerkadresse des lokalen Mac-Betriebs ist nicht als feste IP im Router hinterlegt und koennte sich theoretisch aendern.

Insgesamt wurde das Projekt von der ersten Analyse des bestehenden Excel-Systems bis zu einer eigenstaendigen, mehrfach abgesicherten und oeffentlich (im eingeschraenkten, privaten Sinn) erreichbaren Cloud-Anwendung gefuehrt – mit durchgaengigem Fokus auf Einfachheit fuer die Endnutzer, schrittweiser Umsetzung sowie sorgfaeltiger Absicherung der echten Produktivdaten bei jeder einzelnen Aenderung.